

# SCCAD 3nm PDK

User Manual

Synopsys IC Compiler 2 Edition

|              |                                              |
|--------------|----------------------------------------------|
| Version      | v1.0                                         |
| Release Date | TBD                                          |
| Organization | SCCAD Lab, University of Southern California |
| Tool         | Synopsys IC Compiler 2                       |
| Contact      | sw.jung@gatech (Sungwoo Jung)                |

---

This manual provides comprehensive guidance for users and developers working with the **SCCAD 3nm Process Design Kit (PDK)**. It covers PDK installation from GitHub, environment configuration, and step-by-step instructions for running Place-and-Route (PnR) using **Synopsys IC Compiler 2 (ICC2)**. Additionally, it also provides some tips on how to modify key PDK components — including SPICE model cards and interconnect models (ITF/RC).

---

# Table of Contents

---

## 1 Introduction

- 1.1 Overview of SCCAD 3nm PDK
- 1.2 Key Features and Technology Highlights

## 2 Getting Started — PDK Installation from GitHub

- 2.1 Cloning the Repository
- 2.2 Environment Setup

## 3 Running PnR with Synopsys IC Compiler 2

- 3.1 Essential PDK Files for PnR
- 3.2 PnR Flow Overview

## 4 PDK Development and Modification

- 4.1 Overall Flow of PDK Development
- 4.2 PDK Component Modification
  - 4.2.1 SPICE Model
  - 4.2.2 Interconnect Model (ITF / RC Model)

# 1 Introduction

## 1.1 Overview of SCCAD 3nm PDK

The **SCCAD 3nm Process Design Kit (PDK)** is developed and maintained by the SCCAD Laboratory to support academic and research-oriented digital IC design flows at the 3 nm technology node. The PDK provides a complete set of design enablement files — including technology files, standard-cell libraries, SPICE models, and interconnect models — that allow designers to perform synthesis, place-and-route (PnR), and sign-off verification using industry-standard EDA tools.

Several open-source 3 nm PDKs are available to the research community. The table below positions SCCAD-3nm alongside two other publicly available 3 nm PDKs — FreePDK3 (NCSU) and GT3 (Gatech) — to highlight the distinguishing capabilities of this release.

| Feature                 | FreePDK3 (NCSU)   | GT3 (Gatech) | SCCAD-3nm |
|-------------------------|-------------------|--------------|-----------|
| Tech Node               | 3nm               | 3nm          | 3nm       |
| Device Type             | GAAFET            | GAAFET       | GAAFET    |
| Cell Heights            | 5.5T              | 6T           | 6T, 5T    |
| Power Rail (PR)         | Front-side (FSPR) | Buried (BPR) | FSPR, BPR |
| Support Back-side Metal | No                | No           | Yes       |

As shown in the table, all three PDKs target the 3 nm node and adopt Gate-All-Around FET (GAAFET) device architectures, reflecting the industry transition away from FinFET at this technology generation. Within this shared foundation, the SCCAD-3nm PDK offers several capabilities that extend the design space available to researchers.

- **Dual cell-height support (6T and 5T).** Providing both 6-track and 5-track standard-cell libraries gives designers flexibility to trade off area density against routability, enabling exploration of different optimization targets within a single PDK.
- **Both Front-side and Buried Power Rail (FSPR & BPR).** Supporting both power delivery architectures allows direct comparison of conventional front-side routing against buried power rail approaches — an important research topic at advanced nodes where power grid resistance is a key design constraint.
- **Back-side metal support.** SCCAD-3nm includes back-side metal interconnect, enabling exploration of back-side power delivery network (BS-PDN) and signal routing techniques that are central to next-generation 3D integration research.

Together, these features make SCCAD-3nm well-suited for research spanning standard-cell library characterization, power delivery network analysis, back-side interconnect exploration, and full back-end PnR flows at an advanced technology node.

## 1.2 Key Features and Technology Highlights

### Technology Node: 3 nm GAAFET

The SCCAD-3nm PDK is built on a **3 nm Gate-All-Around FET (GAAFET)** process. Unlike FinFET, where the gate wraps around three sides of a fin, GAAFET surrounds the channel on all four sides using stacked nanosheet or nanowire structures. This architecture provides superior electrostatic control, reduced short-channel effects, and improved drive current at scaled supply voltages — making it the device of choice for sub-5 nm technology nodes. The fabrication process sequence is illustrated in **Figure 1**.

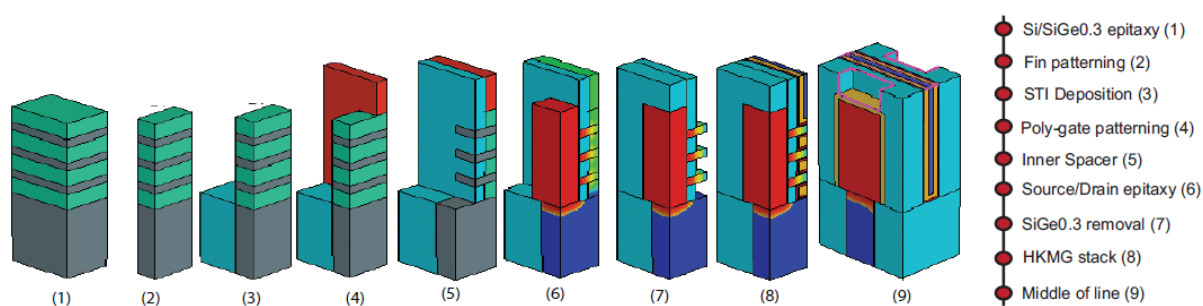


Figure 1. TCAD device structure of the SCCAD 3nm GAAFET.

Table 1 summarizes the key device physical parameters of the SCCAD-3nm NSFET device in comparison with a representative 5 nm FinFET process.

Table 1. Comparison of device physical parameters of FinFET and NSFET device.

| Physical Parameters (nm) | TSMC 5nm | NSFET 3nm |
|--------------------------|----------|-----------|
| CPP                      | 48       | 42        |
| Fin/NS pitch             | 25       | NA        |
| Gate length              | 15       | 12        |
| Spacer length            | 6        | 5         |
| # Fin/NS                 | 2        | 3         |
| Fin/NS thickness         | 6        | 5         |
| Fin/NS width             | NA       | 30, 20    |
| Fin/NS stack height      | 55       | 55        |

### Standard-Cell Library

The SCCAD-3nm PDK includes two standard-cell libraries targeting different design objectives. The **6-Track (6T) library** uses a Front-side Power Rail (FSPR) architecture and offers a balance between performance and density. The **5-Track (5T) library** adopts a Buried Power Rail (BPR) architecture, reclaiming routing resources previously consumed by power stripes and enabling a more compact cell footprint. Together, the two libraries allow designers to explore the PPA trade-space within a single PDK.

The complete cell set comprises **57 standard cells** covering combinational, sequential, and utility functions. Table 2 lists the full cell set along with drive strength variants for each cell type.

| STD Cells    | (Input) x (#parallel Trs)                          |
|--------------|----------------------------------------------------|
| INV          | x1, x2, x3, x4, x5, x6, x7, x8, x10, x12, x14, x16 |
| BUF          | x1, x2, x3, x4, x5, x6, x7, x8, x10                |
| NAND, NOR    | 2x1, 3x1, 4x1, 2x2, 3x2                            |
| AND, OR      | 2x1, 3x1, 4x1                                      |
| XOR, XNOR    | 2x1, 3x1                                           |
| AOI, OAI     | (21, 22, 221, 222, 311, 31) x1                     |
| MUX          | 2x1                                                |
| D Flip-Flop  | Hx1 (async.), HQNx1 (sync.)                        |
| DHL (latch)  | x1                                                 |
| <b>Total</b> | <b>57</b>                                          |

Each cell type in the library serves a distinct role in digital circuit design:

- **INV / BUF** — Inverter and buffer cells are provided in 12 and 9 drive strength variants respectively, covering a wide range of fanout and wire-load scenarios. These cells form the backbone of clock tree synthesis and signal-integrity-driven buffering.
- **NAND / NOR** — 2-, 3-, and 4-input NAND and NOR gates with two parallel transistor options (x1, x2) support both area-optimized and performance-optimized implementations of basic logic.
- **AND / OR** — Non-inverting AND and OR gates in 2-, 3-, and 4-input configurations provide direct implementation of positive logic without requiring an additional inverter stage.
- **XOR / XNOR** — 2- and 3-input exclusive-OR and exclusive-NOR gates are essential for arithmetic datapaths, parity generation, and error-detection logic.
- **AOI / OAI** — And-Or-Invert and Or-And-Invert complex gates in six configurations (21, 22, 221, 222, 311, 31) enable compact, high-speed implementation of multi-level boolean functions in a single cell.
- **MUX** — A 2-to-1 multiplexer cell supports datapath and control-flow selection logic.

- **D Flip-Flop** — Two flip-flop variants are provided: an asynchronous reset type (**Hx1**) and a synchronous reset type with complementary output (**HQNx1**), covering the most common sequential design patterns.
- **DHL (Latch)** — A level-sensitive D-latch is included for time-borrowing and latch-based pipeline designs.

Figure 2 shows the layout of the **INVx1** cell for each library variant. The two libraries differ primarily in cell height, power rail placement, and the resulting routing resource availability.

**6-Track (6T) / Front-Side Power Rail (FSPR):** In the 6T library, the power rails (VDD and VSS) are routed on the **M1** metal layer at the top and bottom of the cell boundary, each rail being 3× the Critical Dimension (CD) wide. The resulting cell height is **144 nm**. The nanosheet width for the GAAFET device is fixed at **30 nm**.

**5-Track (5T) / Buried Power Rail (BPR):** In the 5T library, the power rails are moved to the buried metal layer **MBPR**, connected to the Source/Drain (S/D) epitaxial regions through **VBPR** vias (12 nm × 18 nm). The MBPR metal width is **25 nm**. Burying the power rails below the active device layers reclaims the M1 routing tracks previously occupied by VDD/VSS, reducing the effective cell height to **120 nm** and freeing M1 entirely for signal routing.

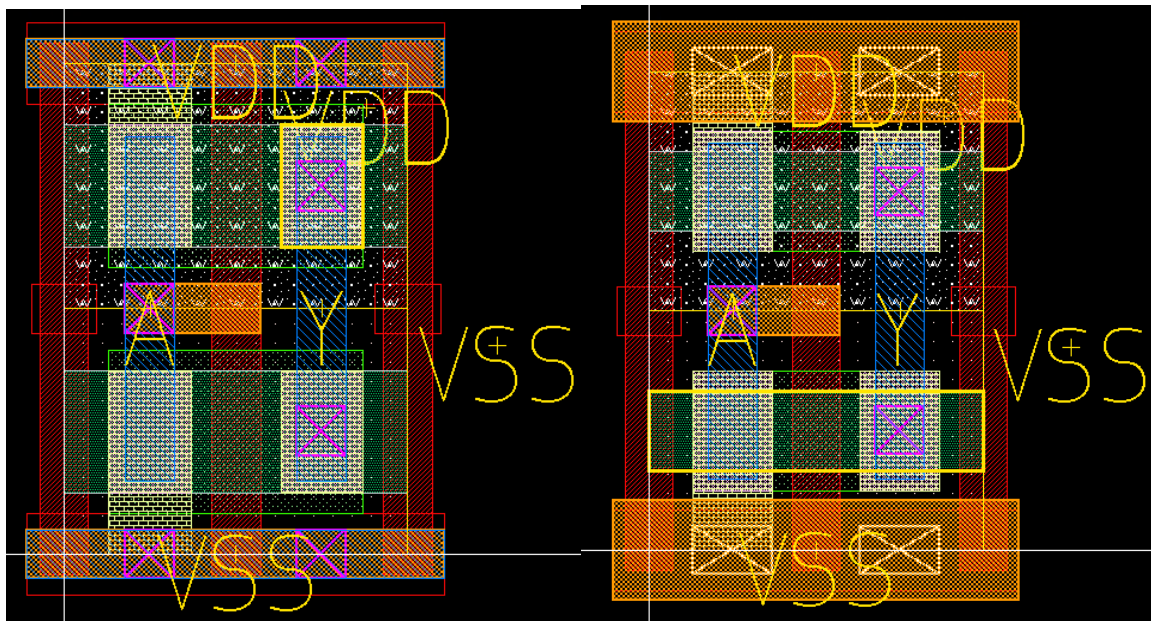


Figure 2a. INVx1 layout — 6T / FSPR library.

Figure 2b. INVx1 layout — 5T / BPR library.

### Back-end-of-Line (BEOL) Technology

The SCCAD-3nm PDK features a comprehensive back-end-of-line (BEOL) stack spanning from the back-side metal layers up through the front-side routing layers. The full interconnect stack consists of metal layers **MB2 through M9** and via layers **VB1 through V8**, organized into three tiers: back-side layers, BPR cell layers, and front-side layers. Table 3 summarizes the key physical parameters of each layer.

|                | Metal      | W (nm)                          | Resistance ( $\Omega$ ) |
|----------------|------------|---------------------------------|-------------------------|
| Back-side      | MB2-MB1    | 61                              | 11                      |
| BPR cell layer | MBPR       | 25                              | 65                      |
|                | M0         | 20                              | 523                     |
| Front-Side     | M1-M3      | 12                              | 347                     |
|                | M4, M5     | 18                              | 101                     |
|                | M6, M7     | 24                              | 44                      |
|                | VIA        | W $\times$ L (nm <sup>2</sup> ) | Resistance ( $\Omega$ ) |
| Back-Side      | VB1        | 40 $\times$ 40                  | 4.0                     |
| BPR cell layer | VBPR       | 20 $\times$ 12                  | 56                      |
| BSP cell layer | $\mu$ -TSV | 60 $\times$ 60                  | 5                       |
|                | BSC        | 20 $\times$ 20                  | 74.9                    |
|                | V0-V3      | 12 $\times$ 12                  | 63.5                    |
| Front-Side     | V4, V5     | 18 $\times$ 18                  | 19.38                   |
|                | V6, V7     | 24 $\times$ 24                  | 10.8                    |

Among the layers listed above, the SCCAD-3nm PDK provides dedicated back-side metal interconnect through two layers: **MB1** and **MB2**. These layers are fabricated on the wafer backside after front-side processing and bonding, and are available for both Power Delivery Network (PDN) routing and general signal routing.

Using MB1 and MB2 for PDN distribution — commonly referred to as a **Back-Side Power Delivery Network (BS-PDN)** — decouples power routing from signal routing, reducing IR drop, improving power integrity, and freeing front-side metal resources for timing-critical signals. In addition, MB1 and MB2 can carry select signal nets where back-side routing offers a wire-length or congestion advantage over front-side alternatives.

### EDA Tool Support

The SCCAD-3nm PDK is validated with the following industry-standard EDA tools. Two separate user manuals are provided — one for the Synopsys flow (this document) and one for the Cadence flow.

## 2 Getting Started — PDK Installation from GitHub

### 2.1 Cloning the Repository

The SCCAD 3nm PDK is hosted on GitHub. Use the following commands to clone the repository to your local workstation or server.

#### HTTPS:

```
git clone https://github.com/sjung366/SCCAD-3nm-PDK.git
cd SCCAD-3nm-PDK
```

#### SSH (recommended for development):

```
git clone git@github.com:SCCAD-Lab/SCCAD-3nm-PDK.git
cd SCCAD-3nm-PDK
```

If the repository uses Git submodules (e.g., for standard-cell libraries or example designs), initialize them as follows:

```
git submodule update --init --recursive
```

### 2.2 Environment Setup

Before running any EDA tool, the following environment variables must be configured manually. These variables define the paths to key PDK components and must be set in your shell environment (e.g., **.bashrc** or **.cshrc**) or exported at the beginning of each session.

#### Required Environment Variables:

```
# Root directory of the PDK repository
export PDK_ROOT=/path/to/sccad-3nm-pdk

# Technology file path (ICC2)
export TECH_PATH=$PDK_ROOT/techfile

# Standard-cell NDM library path
export STDCELL_NDM=$PDK_ROOT/ndm

# Liberty timing library path
export LIB_PATH=$PDK_ROOT/lib

# LEF file path
export LEF_PATH=$PDK_ROOT/lef

# StarRC ITF path
export ITF_PATH=$PDK_ROOT/itf

# RC model path
export RC_PATH=$PDK_ROOT/rc_model
```



After setting the variables, verify that the key PDK files are accessible:

```
# Verify PDK root is set correctly
echo $PDK_ROOT

# Check that essential files exist
ls $TECH_PATH
ls $STDCELL_NDM
ls $LIB_PATH
ls $ITF_PATH
```

### Required EDA Tools:

The following EDA tools must be installed and accessible via your system PATH before running the PnR flow. License servers should be configured according to your institution's setup.

- **Synopsys IC Compiler 2 (ICC2)** — Primary tool for place-and-route. The **icc2\_shell** executable must be accessible from the command line.
- **Synopsys StarRC** — Parasitic extraction tool. Used to generate SPEF files from the routed layout using the PDK ITF and RC models. The **StarXtract** executable must be accessible.
- **Synopsys PrimeTime (PT)** — Static timing analysis and sign-off tool. The **pt\_shell** executable must be accessible.
- **Synopsys Design Compiler (DC)** — Logic synthesis tool for generating the gate-level netlist from RTL. The **dc\_shell** executable must be accessible. (Required for synthesis; optional if a pre-synthesized netlist is provided.)

### System Requirements:

| Item             | Requirement                                          |
|------------------|------------------------------------------------------|
| Operating System | RHEL 7/8, CentOS 7, or compatible Linux (64-bit)     |
| Shell            | bash or csh/tcsh                                     |
| TCL              | Version 8.5 or later (required by ICC2 and PT)       |
| Python           | Version 3.8 or later (required for utility scripts)  |
| Disk Space       | Minimum 10 GB free for PDK files and run directories |
| Memory           | Minimum 32 GB RAM recommended for full PnR runs      |

**License Configuration:** Synopsys tools require a valid license server. Set the license server address using the **SNPSLMD\_LICENSE\_FILE** or **LM\_LICENSE\_FILE** environment variable:

```
export SNPSLMD_LICENSE_FILE=27020@your.license.server
# Verify tool availability
which icc2_shell
which StarXtract
which pt_shell
```

## 3 Running PnR with Synopsys IC Compiler 2

### 3.1 Essential PDK Files for PnR

Before launching a PnR run, it is important to understand which PDK files are loaded at each stage of the flow and what role they play. The SCCAD-3nm PDK is organized into dedicated subdirectories, each containing a specific category of design enablement data. The table below lists the essential files required for a complete ICC2-based PnR run, from initial library setup through parasitic extraction and timing sign-off.

| Directory / File                  | Format                        | Tool      | Description                                          |
|-----------------------------------|-------------------------------|-----------|------------------------------------------------------|
| TF/3nm_GAA_FSPR.tf                | .tf                           | ICC2      | Technology rules: routing layers, vias, design rules |
| TECH-LEF/3nm_GAA_FSPR_tech.lef    | .lef                          | ICC2      | Technology LEF: layer definitions and via rules      |
| LEF/3nm_GAA_FSPR.lef              | .lef                          | ICC2      | Cell LEF: macro dimensions and pin geometry          |
| LIB_DB/3nm_GAA_FSPR_lv_t_nldm.lib | .lib, .db                     | ICC2 / PT | Liberty: timing arcs, power tables                   |
| NDM/3nm_GAA_FSPR.ndm              | .ndm                          | ICC2      | Pre-built ICC2 reference library (NDM format)        |
| ITF/*.itf                         | .itf,<br>.tluplus,<br>.nxtgrd | StarRC    | Interconnect Technology File for RC extraction       |

### 3.2 PnR Flow Overview

The SCCAD-3nm PDK supports a complete RTL-to-GDS back-end flow using Synopsys IC Compiler 2. The overall flow is illustrated in **Figure 3**, spanning from RTL synthesis through physical implementation and PPA analysis. Each stage is described in detail below.

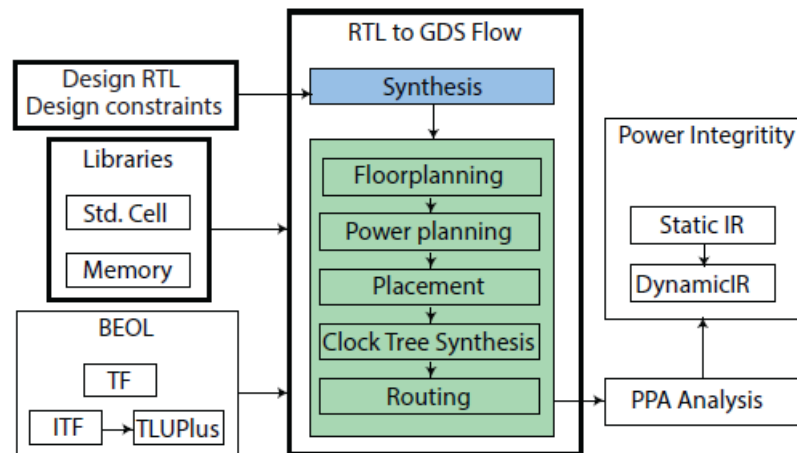


Figure 3. RTL-to-GDS PnR flow with the SCCAD-3nm PDK and Synopsys ICC2.

### Step 1 — RTL Synthesis

The flow begins with logic synthesis using **Synopsys Design Compiler (DC)**. The user provides the RTL source files (Verilog/SystemVerilog) together with design constraints in SDC format, specifying target clock frequency, input/output delays, and load conditions. DC maps the RTL onto the SCCAD-3nm standard-cell library and produces a gate-level netlist along with a synthesized SDC file, which are then passed to ICC2 for physical implementation.

```
dc_shell -f scripts/dc/run_syn.tcl \
-output_log_file logs/syn.log
```

### Step 2 — Design Setup in ICC2

Before physical implementation begins, the design library must be initialized in ICC2 using the **New Data Model (NDM)** format. The pre-built NDM reference library provided in the PDK packages the technology file, LEF, and Liberty data into a single file, eliminating the need to load each component separately. The gate-level netlist and SDC are then read into the design library:

```
# Open or create the design NDM library
create_lib -technology $TECH_PATH/3nm_GAA_FSPR.tf \
-ref_libs $STDCELL_NDM/3nm_GAA_FSPR.ndm \
my_design.ndm

# Read synthesized netlist and constraints
read_verilog results/syn/netlist.v
read_sdc results/syn/constraints.sdc
link_design $TOP_MODULE
```

### Step 3 — Floorplanning and Power Planning

Floorplanning defines the die area, core utilization, and the placement of any hard macros or memory blocks. Power planning follows immediately, creating the VDD/VSS power mesh across the design. For BPR-based designs, the buried power rails are pre-defined in the PDK cell LEF and do

not require explicit strapping at this stage; front-side power rings and stripes are added on top of the BPR network:

```
# Initialize floorplan
initialize_floorplan -utilization 0.70 \
-core_offset {2 2 2 2}

# Create power mesh
create_pg_mesh_pattern -layers {M4 M5} \
-nets {VDD VSS}
compile_pg
```

#### Step 4 — Placement

ICC2 performs timing-driven placement using **place\_opt**, which simultaneously optimizes cell placement, logic restructuring, and buffer insertion to meet timing constraints. At this stage, the tool uses ideal clock assumptions. Users may set target utilization and timing effort level in the run script:

```
place_opt
```

#### Step 5 — Clock Tree Synthesis (CTS)

Clock Tree Synthesis builds the clock distribution network to minimize skew and meet insertion delay targets. ICC2's **clock\_opt** command performs CTS and post-CTS optimization in a single step, using the NDR (Non-Default Routing) rules embedded in the PDK technology file for clock net shielding and spacing:

```
# Run CTS and post-CTS optimization
clock_opt
```

#### Step 6 — Routing

Global and detail routing are performed by **route\_auto**, followed by post-route optimization with **route\_opt**. The router uses the metal stack and spacing rules defined in the PDK technology file. For designs using the back-side metal layers (MB1/MB2), additional routing directives may be required to enable back-side signal or power routing:

```
route_auto
route_opt
```

#### Step 7 — PPA Analysis

After routing and parasitic extraction, the final Power, Performance, and Area (PPA) metrics are evaluated. **Performance** is assessed through static timing analysis (STA) using **Synopsys PrimeTime (PT)**, which reports worst negative slack (WNS) and total negative slack (TNS) across all timing paths. **Power** analysis is performed in PT-PX using switching activity data annotated from post-layout simulation, reporting dynamic and leakage power breakdown by cell type. **Area** is reported directly from ICC2 after routing, covering cell area, net area, and total die utilization. Together these three metrics determine whether the design meets its target specifications and whether further optimization iterations are needed:

```
# Static timing analysis and power analysis – PrimeTime
pt_shell -f scripts/pt/run_sta.tcl \
-output_log_file logs/sta.log

# Area report – ICC2
report_design
report_utilization
```

Upon successful sign-off, the final GDS layout can be streamed out from ICC2 for DRC/LVS verification:

```
write_gds -output results/final/design.gds
```

## 4 PDK Development and Modification

### 4.1 Overall Flow of PDK Development

The SCCAD-3nm PDK was developed through a structured multi-stage flow that integrates TCAD device simulation, physical cell design, parasitic extraction, and library characterization. The full development pipeline is illustrated in **Figure 4**. The yellow-highlighted boxes represent the final PDK deliverables consumed by the PnR flow.

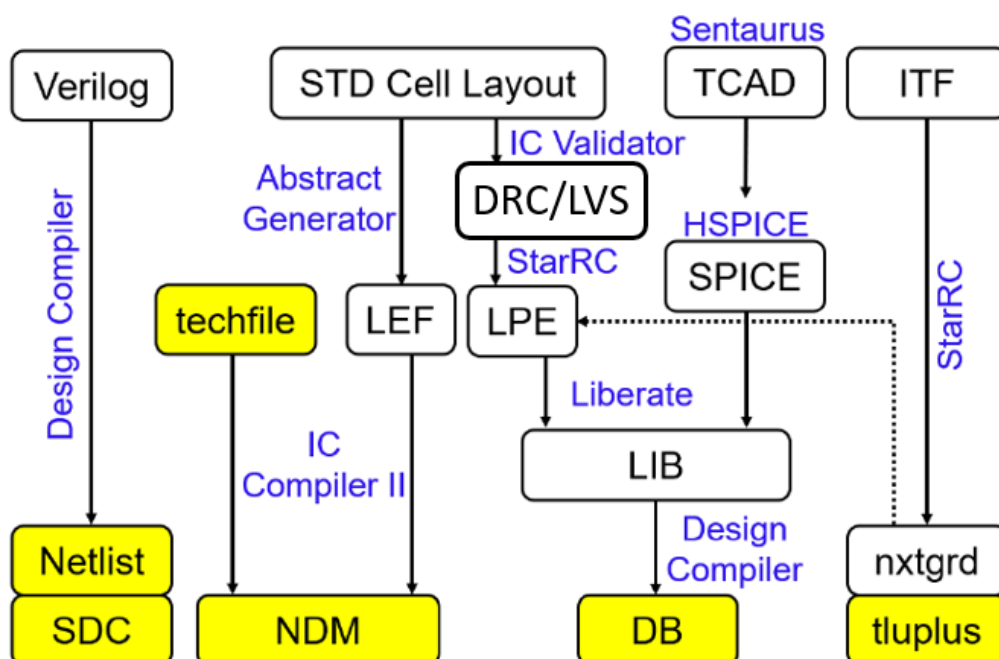


Figure 4. SCCAD-3nm PDK development flow.

### Device Modeling — TCAD to SPICE

The PDK development begins with **TCAD simulation** using Synopsys Sentaurus to characterize the 3nm GAAFET device structure. The simulated device characteristics are fitted to a BSIM-CMG compact model, producing the **SPICE model card** that accurately captures transistor behavior. The SPICE models are subsequently used in two downstream paths: cell-level LPE simulation and Liberty characterization.

#### *Physical Cell Design — Layout to LEF/NDM*

Standard cell layouts are drawn in compliance with the 3nm design rules and verified through **DRC/LVS** using Synopsys IC Validator. Once sign-off clean, an **Abstract Generator** extracts the cell abstract view — pin locations, obstruction layers, and cell dimensions — to produce the **LEF** file. In parallel, the physical layout combined with the technology file (**techfile**) is compiled into the ICC2-native **NDM** reference library using IC Compiler II, which packages all physical views required for PnR into a single database.

#### *Parasitic Extraction — LPE and RC Models*

Layout Parasitic Extraction (**LPE**) is performed on each standard cell using **StarRC**, driven by the ITF interconnect technology file. The extracted parasitic netlist captures the RC contributions of local interconnect within each cell. In addition, StarRC processes the ITF to generate the **nxtgrd** and **TLUPlus** RC tables, which are used during full-chip PnR for fast, field-solver-calibrated wire parasitic estimation across all metal layers.

#### *Library Characterization — LPE to LIB/DB*

Cell timing and power characterization is performed by **Liberate** (Cadence) using HSPICE simulation. Each cell is simulated with the extracted parasitic netlist (LPE) and the corresponding SPICE model card, sweeping input slew and output load conditions. The resulting Non-Linear Delay Model (**NLDM**) data is written out as a **Liberty (.lib)** file. The Liberty file is then compiled into binary **.db** format using Synopsys Design Compiler for use in synthesis and PrimeTime sign-off.

#### *PDK Deliverables*

The outputs of the development flow — **Netlist/SDC** from synthesis, **NDM** for physical implementation, **DB** for timing, and **TLUPlus/nxtgrd** for parasitic extraction — together constitute the complete set of PDK files required to run a full RTL-to-GDS flow with Synopsys ICC2. Any modification to an upstream component (e.g., SPICE model or ITF) requires re-running all downstream steps to ensure consistency across the PDK.

## 4.2 PDK Component Modification

### *4.2.1 SPICE Model*

SPICE model cards define the electrical behavior of transistors used in circuit simulation and cell characterization. The SCCAD-3nm PDK provides HSPICE-format model cards for both NMOS and PMOS devices across five PVT corners. Modifying the SPICE model is necessary when the target process parameters change — for example, when exploring a different threshold voltage flavor,

adjusting the device geometry, or fitting the model to a new set of TCAD simulation data.

#### File location:

```
$PDK_ROOT/model/hspice/
■■■ sccad3nm_tt.l # Typical-Typical (TT) corner
■■■ sccad3nm_ss.l # Slow-Slow (SS) corner
■■■ sccad3nm_ff.l # Fast-Fast (FF) corner
■■■ sccad3nm_fs.l # Fast-Slow (FS) corner
■■■ sccad3nm_sf.l # Slow-Fast (SF) corner
```

#### Challenge: Manual Parameter Tuning

A BSIM-CMG model card for a 3nm GAAFET device contains tens of interdependent parameters — including threshold voltage (**phig**), carrier mobility (**u0**, **ua**, **etamob**), saturation velocity (**vsat**), and short-channel effect coefficients (**eta0**, **pdibl2**, **dvtp0**), among others. Manually tuning each parameter by trial and error to match a target Id-Vgs / Id-Vds characteristic is extremely time-consuming and prone to inconsistency, particularly when fitting across multiple bias conditions simultaneously.

#### Solution: Automated Curve Fitting Script

To address this, the SCCAD-3nm PDK includes an automated curve fitting script (**auto\_curve\_fitting.sp**) that leverages HSPICE's built-in optimization engine (**opt2**) to extract all model parameters simultaneously by minimizing the error between simulated and reference Id-V curves. The reference data — obtained from TCAD simulation or measurement — is supplied as a tabulated data file (**3nm\_Ref\_iv.dat**). The optimizer iterates up to 5000 times with tight convergence tolerances, typically converging within minutes.

#### Script structure: auto\_curve\_fitting.sp

The script defines the device under test with the target geometry:

```
* Device under test – NMOS LVT, W=20nm, L=12nm, nfin=1
m1 drain gate 0 bulk nmos_lvt w=2e-8 l=1.2e-8 nfin=1
```

Each parameter to be optimized is declared with an initial value and a physically meaningful search range using the **opt2()** syntax **opt2(initial, min, max)**:

```
.param
+ phig = opt2(4.5, 4.4, 4.8 ) $ Gate work function (eV)
+ u0 = opt2(0.025145, 0.005, 0.03 ) $ Low-field mobility
+ etamob = opt2(3.0535, 2.7, 3.3 ) $ Mobility degradation exponent
+ vsat = opt2(1.1e+5, 1.0e+5, 1.3e+5) $ Saturation velocity (cm/s)
+ eta0 = opt2(1.3571, 1.0, 3.0 ) $ DIBL coefficient
+ pdibl2 = opt2(6.4836e-3, 2e-3, 1e-2 ) $ DIBL effect parameter
+ dvtp0 = opt2(-1.2334e-2, -0.013, -0.01) $ Vth shift parameter
+ ... $ (16 parameters total)
```

The optimization is driven by minimizing the RMS error between simulated current **i(m1)** and the reference data **par(ids)** loaded from the external file:

```
* Load reference IV data (TCAD or measurement)
.data iv merge
+ file=3nm_Ref_iv.dat vds=1 vgs=2 ids=3
.enddata

* Run optimization – up to 5000 iterations, tight convergence
.dc data=iv optimize=opt2 results=compl2 model=optmod2
.model optmod2 opt itropt=5000 relout=1e-6 relin=1e-8 close=10

* Error metric: RMS error between reference and simulated Id
.measure dc compl2 err2 par(ids) i(m1) minval=1e-11 ignore=1e-12
```

### How to use the script:

- Step 1: Prepare the reference IV data file **3nm\_Ref\_iv.dat** with columns [Vds, Vgs, Ids] from your TCAD simulation or target specification.
- Step 2: Adjust the initial values and search ranges in the **.param** block if needed, based on prior knowledge of the target process.
- Step 3: Run the script with HSPICE: **hspice auto\_curve\_fitting.sp > fitting.log**
- Step 4: The optimized parameter values are printed in the log file. Copy the extracted values into the target corner model card (e.g., **sccad3nm\_tt.l**).
- Step 5: Verify the fitted model by re-running the simulation without optimization and comparing the Id-Vgs and Id-Vds curves against the reference data.
- Step 6: If the change affects cell timing or power, re-run Liberty characterization and recompile the **.db** file before the next PnR run.

### 4.2.2 Interconnect Model (ITF / RC Model)

The Interconnect Technology File (**ITF**) describes the physical properties of the metal stack — layer thicknesses, sheet resistances, via resistances, and dielectric constants — used by StarRC to compute parasitic resistance and capacitance for every net in the design. Accurate RC models are critical for timing convergence at 3 nm, where interconnect delay can dominate gate delay on longer wires. ITF modification is necessary when exploring alternative metal stack configurations, modeling process variation, or calibrating the PDK to a new foundry target.

#### File location:

```
$PDK_ROOT/itf/
■■■ sccad3nm.itf # Main Interconnect Technology File
■■■ sccad3nm.tluplus # TLUPlus RC table (used by ICC2)
■■■ sccad3nm.nxtgrd # nxtgrd RC table (used by StarRC)
```

#### ITF key parameters:

```
ILAYER M1
RTHICK = [value] ; Metal thickness (nm)
RHO = [value] ; Sheet resistance (Ω/sq)
WMIN = [value] ; Minimum wire width (nm)
```



```

...

ILAYER VIA1
RPV = [value] ; Via resistance ( $\Omega$ )
...

IDIELECTRIC
ER = [value] ; Relative permittivity (k)
...

```

### Challenge: Consistent Resistance Scaling

A full 3nm BEOL stack contains metal layers (M0–M9, MBPR, MB1–MB2) and via layers (V0–V8, VBPR, VB1,  $\mu$ -TSV, BSC), each with its own resistance entry in the ITF. When a process change requires scaling resistance values — for example, modeling increased metal resistivity due to grain boundary scattering at narrow line widths, or evaluating the impact of a  $\pm 10\%$  resistance variation — manually editing every **RHO**, **RPV**, and **VALUES** entry across the file is tedious and error-prone.

### Solution: Automated ITF Resistance Scaling Script

The SCCAD-3nm PDK provides a Python utility script (**auto\_itf\_scaling.py**) that automates resistance scaling across the entire ITF file in a single command. The script uses regular expressions to locate and update all resistance-related entries — via resistance (**RPV**), sheet resistance (**RHO**), and pre-computed resistance tables (**VALUES**) — and multiplies each by a user-specified scale factor. Unrelated parameters such as layer thickness and dielectric constant are left unchanged.

#### Script structure: auto\_itf\_scaling.py

The script accepts four command-line arguments and supports selective targeting of via resistance, metal sheet resistance, or table values:

```

python auto_itf_scaling.py \
-i sccad3nm.itf \ # Input ITF file
-o sccad3nm_scaled.itf \ # Output ITF file
-s 1.5 \ # Scale factor (e.g., 1.5x)
-t all \ # Target: via | metal | table | all

```

Internally, the script applies three independent regex-based replacements. For **via resistance** (RPV) and **sheet resistance** (RHO), the numeric value is extracted and multiplied by the scale factor:

```

# Scale via resistance: RPV=value → RPV=value*scale
if ('via' in targets or scale_all) and 'RPV=' in line:
    line = re.sub(r'RPV=([0-9\.eE+-]+)',
    lambda m: f"RPV={float(m.group(1))*scale_factor:.6g}",
    line)

# Scale sheet resistance: RHO=value → RHO=value*scale
if ('metal' in targets or scale_all) and 'RHO=' in line:

```

```
line = re.sub(r'RHO=([0-9\.eE+-]+)',
lambda m: f'RHO={float(m.group(1))*scale_factor:.6g}',
line)
```

For resistance **table entries** (VALUES), the entire space-separated value list within the braces is parsed, scaled, and written back:

```
# Scale resistance table: VALUES { v1 v2 ... } → VALUES { v1*s v2*s ... }
if ('table' in targets or scale_all) and 'VALUES' in line and '{' in line:
line = re.sub(
r'VALUES\s*\s*(.*?)\s*\}',
lambda m: "VALUES { " +
"\t".join(f"{float(v)*scale_factor:.6g}"
for v in m.group(1).split()) + " }",
line)
```

### How to use the script:

- Step 1: Identify the resistance parameters to modify. Use **-t via** to scale only via resistance, **-t metal** for sheet resistance, **-t table** for pre-computed RC table values, or **-t all** to scale all resistance entries simultaneously.
- Step 2: Run the script with the desired scale factor. For example, to model a 10% increase in metal resistance: **python auto\_itf\_scaling.py -i sccad3nm.itf -o sccad3nm\_110pct.itf -s 1.1 -t metal**
- Step 3: Verify the output file by checking that RHO and/or RPV values have been updated correctly while layer thicknesses and dielectric constants remain unchanged.
- Step 4: Re-generate the TLUPlus and nxtgrd RC tables from the updated ITF using StarRC: **StarXtract -cmd scripts/starrc/gen\_tluplus.cmd**
- Step 5: Re-run the full PnR flow with the new RC tables and compare PPA results against the baseline to evaluate the impact of the resistance change.